

Universitatea Politehnica Timișoara
Facultatea de Automatică și Calculatoare

Programare .NET

Îndrumător de laborator



ș.l. dr. ing. Valer BOCAN, CSSLP

E-mail: valer.bocan@upt.ro

Web: www.bocan.ro

Cuprins

Laboratorul 1: Introducere în platforma .NET.....	7
Obiective	7
Activități	7
Configurarea mediului de dezvoltare	7
Crearea primelor programe C# și F#	7
Procesul de compilare și execuție	9
Teme pentru acasă	9
Laboratorul 2: Bazele limbajului C# (I)	11
Obiective	11
Activități	11
Tipuri de date și variabile	11
Operatori și expresii	12
Structuri de control - if și switch	13
Structuri repetitive - while, do-while, for.....	14
Teme pentru acasă	15
Laboratorul 3: Bazele limbajului C# (II)	16
Obiective	16
Activități	16
Definirea și apelarea metodelor	16
Parametri și valori returnate	17
Supraîncărcarea metodelor	18
Implementarea algoritmilor simpli	19
Teme pentru acasă	20
Laboratorul 4: Programare orientată pe obiecte în C#	21
Obiective	21
Activități	21
Definirea și utilizarea claselor și obiectelor	21
Proprietăți și metode	22
Constructorii și modificatori de acces	23
Implementarea unui model simplu de clasă.....	24
Teme pentru acasă	25
Laboratorul 5: Colecții și LINQ în C#	26

Obiective	26
Activități	26
Tablouri și liste	26
Dicționare	27
Interogări LINQ	28
Manipularea datelor cu LINQ	29
Teme pentru acasă	31
Laboratorul 6: Gestionarea erorilor în C#	32
Obiective	32
Activități	32
Prinderea și tratarea excepțiilor cu try-catch	32
Aruncarea și propagarea excepțiilor.....	33
Crearea și utilizarea excepțiilor personalizate	34
Aplicarea bunelor practici în gestionarea erorilor	35
Teme pentru acasă	36
Laboratorul 7: Prelucrarea fișierelor CSV și XML cu LINQ în C#	38
Obiective	38
Activități	38
Prelucrarea fișierelor CSV cu LINQ	38
Prelucrarea fișierelor XML cu LINQ	40
Teme pentru acasă	42
Laboratorul 8: Introducere în F#	43
Obiective	43
Activități	43
Sintaxa de bază F#	43
Tipuri de date în F#	44
Funcții în F#	45
Teme pentru acasă	46
Laboratorul 9: Programare funcțională în F#.....	47
Obiective	47
Activități	47
Pattern matching.....	47

Recursivitate	48
Funcții de ordin superior	49
Teme pentru acasă	50
Laboratorul 10: Tipuri de date în F#.....	52
Obiective	52
Activități	52
Tipuri record	52
Tipuri discriminate (discriminated unions)	53
Tipul Option	54
Teme pentru acasă	55
Laboratorul 11: Colecții și prelucrarea datelor în F#	56
Obiective	56
Activități	56
Liste și secvențe	56
Operații pe colecții	57
List comprehensions	58
Map și Set.....	59
Teme pentru acasă	60
Laboratorul 12: Gestionarea erorilor în F#	62
Obiective	62
Activități	62
Railway-oriented programming (ROP)	62
Computation Expressions.....	63
Validarea datelor	65
Teme pentru acasă	66
Laboratorul 13: Prelucrarea fișierelor CSV și XML cu F#.....	68
Obiective	68
Activități	68
Prelucrarea fișierelor CSV cu funcții din biblioteca standard.....	68
Prelucrarea fișierelor XML cu biblioteca FSharp.Data.....	69
Teme pentru acasă	71
Laboratorul 14: Interoperabilitate C# și F#	72

Obiective	72
Activități	72
Utilizarea tipurilor C# în F#	72
Expunerea tipurilor F# către C#	73
Dezvoltarea de componente mixte C# și F#	75
Teme pentru acasă	77

Laboratorul 1: Introducere în platforma .NET

Obiective

În cadrul acestui laborator, studenții vor:

- Configura mediul de dezvoltare Visual Studio Code pentru programarea în C# și F#
- Instala extensiile necesare pentru C# și F# în Visual Studio Code
- Crea primele programe simple în C# și F# pentru a se familiariza cu sintaxa de bază
- Înțelege procesul de compilare și execuție a programelor .NET
- Utiliza Git pentru managementul codului sursă

Activități

Configurarea mediului de dezvoltare

În această activitate veți instala și configura mediul de dezvoltare Visual Studio Code și extensiile necesare pentru a putea crea aplicații .NET în C# și F#.

Pașii de urmat:

1. Descărcați și instalați Visual Studio Code de la adresa: <https://code.visualstudio.com/download>
2. Lansați Visual Studio Code și deschideți tab-ul Extensions (Ctrl+Shift+X).
3. Căutați și instalați următoarele extensii:
 - a. C# de Microsoft
 - b. Ionide-fsharp de Ionide
4. Reporniți Visual Studio Code pentru a activa extensiile instalate.
5. Deschideți paleta de comenzi (Ctrl+Shift+P) și executați comanda ".NET: Install .NET SDK" pentru a instala SDK-ul .NET necesar.

Crearea primelor programe C# și F#

În această activitate veți crea și rula primele dumneavoastră programe simple în C# și F# pentru a vă familiariza cu sintaxa de bază și structura unui program.

Pașii de urmat pentru C#:

1. Creați un folder nou numit "PrimulMeuProgramC".
2. Deschideți folderul în Visual Studio Code (File > Open Folder).
3. Creați un fișier nou numit "Program.cs" în folderul deschis.
4. Adăugați următorul cod în fișierul "Program.cs":

```
using System;
```

```
namespace PrimulMeuProgramC
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("Salut din C#!");
        }
    }
}
```

5. Deschideți terminalul integrat în Visual Studio Code (Terminal > New Terminal).
6. În terminal, navigați în folderul "PrimulMeuProgramC" și rulați următoarele comenzi pentru a compila și executa programul:

```
dotnet build
dotnet run
```

7. Observați rezultatul în terminalul integrat.
8. Modificați programul pentru a citi numele utilizatorului de la tastatură și pentru a afișa un mesaj personalizat. Utilizați următorul cod:

```
Console.Write("Introduceti numele: ");
string nume = Console.ReadLine();
Console.WriteLine($"Salut, {nume}! Bine ati venit in C#!");
```

9. Salvați fișierul (Ctrl+S) și rulați din nou programul folosind **dotnet run**. Introduceți numele dumneavoastră când este cerut și verificați dacă programul afișează mesajul personalizat.

Pași de urmat pentru F#:

1. Creați un folder nou numit "PrimulMeuProgramF".
2. Deschideți folderul în Visual Studio Code.
3. Creați un fișier nou numit "Program.fs" în folderul deschis.
4. Adăugați următorul cod în fișierul "Program.fs":

```
printfn "Salut din F#!"
```

5. Deschideți terminalul integrat în Visual Studio Code.
6. În terminal, navigați în folderul "PrimulMeuProgramF" și rulați următoarele comenzi pentru a compila și executa programul:

```
dotnet build
dotnet run
```

7. Observați rezultatul în terminalul integrat.
8. Modificați programul pentru a citi numele utilizatorului de la tastatură și pentru a afișa un mesaj personalizat. Utilizați următorul cod:


```
printf "Introduceti numele: "  
let nume = Console.ReadLine()  
printfn $"Salut, {nume}! Bine ati venit in F#!"
```

9. Salvați fișierul (Ctrl+S) și rulați din nou programul folosind **dotnet run**. Introduceți numele dumneavoastră când este cerut și verificați dacă programul afișează mesajul personalizat.

Procesul de compilare și execuție

În această activitate veți explora procesul de compilare și execuție a programelor .NET folosind Visual Studio Code și utilitarul .NET CLI.

Pașii de urmat:

1. Deschideți folderul "PrimulMeuProgramC" în Visual Studio Code.
2. În terminalul integrat, navigați în folderul proiectului.
3. Executați comanda **dotnet build** pentru a compila proiectul. Observați fișierele generate în folderul "bin/Debug".
4. Executați comanda **dotnet run** pentru a rula programul compilat. Observați outputul în terminal.
5. Efectuați modificări în codul sursă al programului (de ex. schimbați mesajul afișat) și salvați fișierul.
6. Executați din nou comanda **dotnet run** și observați că modificările nu sunt reflectate. Aceasta se datorează faptului că trebuie să recompilați programul pentru ca modificările să fie aplicate.
7. Executați comanda **dotnet build** pentru a recompila proiectul cu modificările efectuate.
8. Rulați iar programul folosind **dotnet run** și observați că modificările sunt acum prezente.

Repetăți pașii similari și pentru proiectul F#.

Teme pentru acasă

1. Modificați programele C# și F# de la punctul 2 pentru a calcula suma a două numere introduse de utilizator. Afișați rezultatul sub forma: "Suma numerelor <a> și este <suma>.", unde <a>, și <suma> vor fi înlocuite cu valorile efective.
2. Creați un program C# care afișează versiunea curentă a .NET SDK folosită. (Indiciu: utilizați comanda **dotnet --version** în terminal).
3. Scrieți un program C# care cere utilizatorului să introducă raza unui cerc și calculează și afișează aria și circumferința cercului, folosind valorile introduse. (Considerați $\pi = 3.1415926$)
4. Creați un program F# care convertește o temperatură introdusă de utilizator din grade Celsius în grade Fahrenheit, folosind formula: **fahrenheit = celsius * 9 / 5 + 32**.

-
5. (Mediu) Scrieți un program C# care cere utilizatorului să introducă două șiruri de caractere și verifică dacă primul șir apare în al doilea șir. Afișați un mesaj corespunzător.

Laboratorul 2: Bazele limbajului C# (I)

Obiective

În cadrul acestui laborator, studenții vor:

- Înțelege și utiliza diferite tipuri de date în C#
- Declara și inițializa variabile
- Efectua operații aritmetice, relaționale și logice folosind operatori
- Utiliza structuri de control precum if, switch și bucle (while, do-while, for)
- Rezolva probleme simple folosind elementele de bază ale limbajului C#

Activități

Tipuri de date și variabile

În această activitate veți explora tipurile de date fundamentale în C# și veți învăța cum să declarați și să inițializați variabile.

Pașii de urmat:

1. Deschideți Visual Studio Code și creați un folder nou pentru laboratorul 2.
2. Creați un fișier nou numit "TipuriDeDate.cs" în folderul laboratorului.
3. Adăugați următorul cod în fișier pentru a declara și inițializa variabile de diferite tipuri:

```
using System;

namespace TipuriDeDate
{
    class Program
    {
        static void Main(string[] args)
        {
            int numarIntreg = 42;
            float numarReal = 3.14f;
            double numarPrecizie = 2.71828;
            bool valoareBooleana = true;
            char caracter = 'A';
            string sir = "Hello, C#!";

            Console.WriteLine($"Numar intreg: {numarIntreg}");
            Console.WriteLine($"Numar real: {numarReal}");
            Console.WriteLine($"Numar precizie dubla:
{numarPrecizie}");
            Console.WriteLine($"Valoare booleana:
{valoareBooleana}");
```

```
        Console.WriteLine($"Caracter: {caracter}");  
        Console.WriteLine($"Sir de caractere: {sir}");  
    }  
}
```

4. Salvați fișierul și rulați programul folosind comanda **dotnet run** în terminal. Observați outputul generat.
5. Modificați valorile variabilelor și rulați din nou programul pentru a observa efectele.
6. Adăugați o nouă variabilă de tip **decimal** și inițializați-o cu o valoare. Afișați valoarea ei în consolă.

Operatori și expresii

În această activitate veți utiliza diferiți operatori pentru a efectua operații aritmetice, relaționale și logice în C#.

Pașii de urmat:

1. Creați un fișier nou numit "Operatori.cs" în folderul laboratorului.
2. Adăugați următorul cod în fișier:

```
using System;  
  
namespace Operatori  
{  
    class Program  
    {  
        static void Main(string[] args)  
        {  
            int a = 10;  
            int b = 3;  
  
            // Operatori aritmetici  
            int suma = a + b;  
            int diferenta = a - b;  
            int produs = a * b;  
            int cat = a / b;  
            int restul = a % b;  
  
            Console.WriteLine($"Suma: {suma}");  
            Console.WriteLine($"Diferenta: {diferenta}");  
            Console.WriteLine($"Produs: {produs}");  
            Console.WriteLine($"Cat: {cat}");  
            Console.WriteLine($"Restul: {restul}");  
  
            // Operatori relationali
```

```

        bool egalitate = a == b;
        bool diferit = a != b;
        bool maiMare = a > b;
        bool maiMicSauEgal = a <= b;

        Console.WriteLine($"Egalitate: {egalitate}");
        Console.WriteLine($"Diferit: {diferit}");
        Console.WriteLine($"Mai mare: {maiMare}");
        Console.WriteLine($"Mai mic sau egal:
{maiMicSauEgal}");

        // Operatori logici
        bool siLogic = (a > 5) && (b < 5);
        bool sauLogic = (a > 5) || (b < 5);
        bool negatie = !(a == b);

        Console.WriteLine($"SI logic: {siLogic}");
        Console.WriteLine($"SAU logic: {sauLogic}");
        Console.WriteLine($"Negatie: {negatie}");
    }
}

```

3. Rulați programul și observați rezultatele operațiilor efectuate.
4. Modificați valorile variabilelor **a** și **b** și rulați din nou programul pentru a vedea cum afectează acestea rezultatele.
5. Adăugați un nou set de operații folosind operatorul ternar (**?:**) pentru a determina valoarea maximă dintre **a** și **b**. Afișați rezultatul.

Structuri de control - if și switch

În această activitate veți utiliza structurile de control **if** și **switch** pentru a controla fluxul de execuție al programului bazat pe anumite condiții.

Pașii de urmat:

1. Creați un fișier nou numit "StructuriDeControl.cs" în folderul laboratorului.
2. Adăugați următorul cod în fișier:

```

using System;

namespace StructuriDeControl
{
    class Program
    {
        static void Main(string[] args)
        {
            int numar = 42;

```

```
// Structura if-else
if (numar % 2 == 0)
{
    Console.WriteLine($"{numar} este par.");
}
else
{
    Console.WriteLine($"{numar} este impar.");
}

// Structura switch
switch (numar)
{
    case 10:
        Console.WriteLine("Numarul este 10.");
        break;
    case 20:
        Console.WriteLine("Numarul este 20.");
        break;
    default:
        Console.WriteLine($"Numarul este diferit de
10 si 20. Este {numar}.");
        break;
}
}
}
```

3. Rulați programul și observați rezultatele.
4. Modificați valoarea variabilei **numar** și rulați din nou programul pentru a vedea cum afectează aceasta execuția blocurilor **if** și **switch**.
5. Adăugați o nouă structură **if-else** care verifică dacă numărul este pozitiv, negativ sau zero și afișează un mesaj corespunzător.

Structuri repetitive - while, do-while, for

În această activitate veți utiliza buclele **while**, **do-while** și **for** pentru a executa repetat un bloc de cod.

Pașii de urmat:

1. Creați un fișier nou numit "BucleRepetitive.cs" în folderul laboratorului.
2. Adăugați următorul cod în fișier:

```
using System;

namespace BucleRepetitive
```

```
{
    class Program
    {
        static void Main(string[] args)
        {
            // Bucla while
            int i = 0;
            while (i < 5)
            {
                Console.WriteLine($"Valoarea lui i este: {i}");
                i++;
            }

            // Bucla do-while
            int j = 0;
            do
            {
                Console.WriteLine($"Valoarea lui j este: {j}");
                j++;
            } while (j < 5);

            // Bucla for
            for (int k = 0; k < 5; k++)
            {
                Console.WriteLine($"Valoarea lui k este: {k}");
            }
        }
    }
}
```

3. Rulați programul și observați outputul generat de fiecare buclă.
4. Modificați condițiile de oprire ale buclelor și rulați din nou programul pentru a vedea cum afectează aceasta numărul de iterații.
5. Adăugați o nouă buclă **for** care calculează suma numerelor de la 1 la 100 și afișează rezultatul.

Teme pentru acasă

1. Scrieți un program care citește trei numere de la tastatură și afișează cel mai mare dintre ele.
2. Creați un program care verifică dacă un an introdus de utilizator este bisect.
3. Scrieți un program care generează tabla înmulțirii pentru un număr dat de la tastatură.
4. Creați un program care calculează suma cifrelor unui număr introdus de utilizator.
5. Scrieți un program care afișează toate numerele prime mai mici decât un număr dat de la tastatură.

Laboratorul 3: Bazele limbajului C# (II)

Obiective

În cadrul acestui laborator, studenții vor:

- Defini și utiliza metode în C#
- Înțelege și folosi parametri și valori returnate
- Implementa supraîncărcarea metodelor
- Rezolva probleme simple folosind metode și algoritmi de bază

Activități

Definirea și apelarea metodelor

În această activitate veți învăța cum să definiți și să apelați metode în C#.

Pași de urmat:

1. Creați un fișier nou numit "Metode.cs" în folderul laboratorului.
2. Adăugați următorul cod în fișier pentru a defini și apela o metodă simplă:

```
using System;

namespace Metode
{
    class Program
    {
        static void Main(string[] args)
        {
            AfiseazaMesaj();
        }

        static void AfiseazaMesaj()
        {
            Console.WriteLine("Salut din metoda  
AfiseazaMesaj!");
        }
    }
}
```

3. Rulați programul și observați outputul generat de apelul metodei **AfiseazaMesaj()**.
4. Adăugați o nouă metodă numită **AfiseazaSuma** care primește doi parametri întregi, calculează suma lor și o afișează în consolă.
5. Apelați metoda **AfiseazaSuma** din **Main** cu diferite valori pentru parametri și observați rezultatele.

Parametri și valori returnate

În această activitate veți explora utilizarea parametrilor și a valorilor returnate în metodele C#.

Pașii de urmat:

1. Creați un fișier nou numit "ParametriSiValoriReturnate.cs" în folderul laboratorului.
2. Adăugați următorul cod în fișier:

```
using System;

namespace ParametriSiValoriReturnate
{
    class Program
    {
        static void Main(string[] args)
        {
            int suma = CalculeazaSuma(10, 20);
            Console.WriteLine($"Suma este: {suma}");

            int produs;
            bool succes = InmultesteSiVerifica(3, 4, out
produs);
            if (succes)
            {
                Console.WriteLine($"Produsul este: {produs}");
            }
            else
            {
                Console.WriteLine("Unul dintre numere este
zero!");
            }
        }

        static int CalculeazaSuma(int a, int b)
        {
            return a + b;
        }

        static bool InmultesteSiVerifica(int a, int b, out int
rezultat)
        {
            rezultat = a * b;
            return (a != 0 && b != 0);
        }
    }
}
```

```
}
```

3. Analizați codul și observați cum sunt utilizați parametrii și valorile returnate în metodele `CalculeazaSuma` și `InmultesteSiVerifica`.
4. Rulați programul și urmăriți outputul generat.
5. Modificați metoda `InmultesteSiVerifica` pentru a returna `rezultat` prin valoarea de retur a metodei în loc de parametrul `out`.

Supraîncărcarea metodelor

În această activitate veți învăța despre supraîncărcarea metodelor în C#.

Pași de urmat:

1. Creați un fișier nou numit "SupraincercareaMetodelor.cs" în folderul laboratorului.
2. Adăugați următorul cod în fișier:

```
using System;

namespace SupraincercareaMetodelor
{
    class Program
    {
        static void Main(string[] args)
        {
            AfiseazaDetalii("John Doe");
            AfiseazaDetalii("Jane Doe", 25);
            AfiseazaDetalii("Alice Smith", 30,
"alice@example.com");
        }

        static void AfiseazaDetalii(string nume)
        {
            Console.WriteLine($"Nume: {nume}");
        }

        static void AfiseazaDetalii(string nume, int varsta)
        {
            Console.WriteLine($"Nume: {nume}, Varsta:
{varsta}");
        }

        static void AfiseazaDetalii(string nume, int varsta,
string email)
        {
            Console.WriteLine($"Nume: {nume}, Varsta: {varsta},
Email: {email}");
        }
    }
}
```

```
}  
}  
}
```

3. Observați cum sunt definite trei versiuni ale metodei **AfiseazaDetalii**, fiecare cu un număr diferit de parametri.
4. Rulați programul și urmăriți outputul generat de apelurile către diferitele versiuni ale metodei **AfiseazaDetalii**.
5. Adăugați o nouă versiune a metodei **AfiseazaDetalii** care primește un singur parametru de tip **int** reprezentând vârsta și afișează un mesaj corespunzător.

Implementarea algoritmilor simpli

În această activitate veți implementa câțiva algoritmi simpli folosind metode în C#.

Pașii de urmat:

1. Creați un fișier nou numit "Algoritmi.cs" în folderul laboratorului.
2. Adăugați următoarele metode în fișier:

```
static int SumaNumere(int n)
{
    int suma = 0;
    for (int i = 1; i <= n; i++)
    {
        suma += i;
    }
    return suma;
}

static long Factorial(int n)
{
    if (n == 0)
    {
        return 1;
    }
    else
    {
        return n * Factorial(n - 1);
    }
}

static bool EstePrim(int n)
{
    if (n < 2)
    {
        return false;
    }
}
```

```
for (int i = 2; i <= Math.Sqrt(n); i++)  
{  
    if (n % i == 0)  
    {  
        return false;  
    }  
}  
return true;  
}
```

3. În metoda **Main**, apelați fiecare dintre metodele definite cu diferite valori pentru parametri și afișați rezultatele.
4. Analizați implementarea fiecărei metode și asigurați-vă că înțelegeți cum funcționează algoritmi.
5. Adăugați o nouă metodă care calculează al n-lea număr din șirul lui Fibonacci folosind recursivitate.

Teme pentru acasă

1. Scrieți o metodă care primește un șir de caractere și returnează numărul de vocale din șir.
2. Creați o metodă care primește un număr întreg și returnează inversul său (ex: 123 -> 321).
3. Implementați o metodă care primește un tablou de întregi și returnează valoarea minimă și valoarea maximă din tablou.
4. Scrieți o metodă care primește un număr întreg și returnează suma divizorilor săi.
5. Creați o metodă care primește un tablou de întregi și un număr țintă și returnează indicii a două elemente din tablou a căror sumă este egală cu ținta.

Laboratorul 4: Programare orientată pe obiecte în C#

Obiective

În cadrul acestui laborator, studenții vor:

- Înțelege conceptele de bază ale programării orientate pe obiecte (OOP)
- Defini și utiliza clase și obiecte în C#
- Implementa proprietăți, metode și constructori în clasele C#
- Aplica principiile de encapsulare și utilizarea modificatorilor de acces
- Rezolva probleme simple folosind abordarea OOP

Activități

Definirea și utilizarea claselor și obiectelor

În această activitate veți învăța cum să definiți și să utilizați clase și obiecte în C#.

Pași de urmat:

1. Creați un fișier nou numit "Person.cs" în folderul laboratorului.
2. Adăugați următorul cod în fișier pentru a defini clasa **Person**:

```
class Person
{
    public string Name;
    public int Age;

    public void SayHello()
    {
        Console.WriteLine($"Hello, my name is {Name} and I am {Age} years old.");
    }
}
```

3. Creați un fișier nou numit "Program.cs" în același folder și adăugați următorul cod:

```
using System;

class Program
{
    static void Main(string[] args)
    {
        Person person1 = new Person();
        person1.Name = "John";
        person1.Age = 25;
        person1.SayHello();
    }
}
```

```
        Person person2 = new Person();  
        person2.Name = "Alice";  
        person2.Age = 30;  
        person2.SayHello();  
    }  
}
```

4. Rulați programul și observați outputul generat.
5. Modificați clasa **Person** pentru a adăuga o nouă proprietate **Email** și o metodă **SendEmail** care afișează un mesaj cu adresa de email a persoanei.

Proprietăți și metode

În această activitate veți explora utilizarea proprietăților și metodelor în clasele C#.

Pași de urmat:

1. Creați un fișier nou numit "Rectangle.cs" în folderul laboratorului.
2. Adăugați următorul cod în fișier:

```
class Rectangle  
{  
    private double length;  
    private double width;  
  
    public double Length  
    {  
        get { return length; }  
        set { length = value; }  
    }  
  
    public double Width  
    {  
        get { return width; }  
        set { width = value; }  
    }  
  
    public double GetArea()  
    {  
        return length * width;  
    }  
  
    public double GetPerimeter()  
    {  
        return 2 * (length + width);  
    }  
}
```

3. În fișierul "Program.cs", adăugați următorul cod în metoda **Main**:

```
Rectangle rectangle = new Rectangle();
rectangle.Length = 5;
rectangle.Width = 3;
Console.WriteLine($"Area: {rectangle.GetArea()}");
Console.WriteLine($"Perimeter: {rectangle.GetPerimeter()}");
```

4. Rulați programul și observați outputul generat.
5. Modificați clasa **Rectangle** pentru a adăuga o metodă **Scale** care primește un factor de scalare și actualizează lungimea și lățimea dreptunghiului în consecință.

Constructori și modificatori de acces

În această activitate veți învăța despre constructori și modificatori de acces în C#.

Pași de urmat:

1. Creați un fișier nou numit "Car.cs" în folderul laboratorului.
2. Adăugați următorul cod în fișier:

```
class Car
{
    private string brand;
    private string model;
    private int year;

    public Car(string brand, string model, int year)
    {
        this.brand = brand;
        this.model = model;
        this.year = year;
    }

    public void DisplayInfo()
    {
        Console.WriteLine($"Brand: {brand}, Model: {model}, Year: {year}");
    }
}
```

3. În fișierul "Program.cs", adăugați următorul cod în metoda **Main**:

```
Car car = new Car("Toyota", "Camry", 2020);
car.DisplayInfo();
```

4. Rulați programul și observați outputul generat.

5. Modificați clasa **Car** pentru a adăuga un constructor fără parametri care inițializează proprietățile cu valori implicite.

Implementarea unui model simplu de clasă

În această activitate veți implementa un model simplu de clasă pentru a reprezenta un cont bancar.

Pași de urmat:

1. Creați un fișier nou numit "BankAccount.cs" în folderul laboratorului.
2. Adăugați următorul cod în fișier:

```
class BankAccount
{
    private string accountNumber;
    private string accountHolder;
    private decimal balance;

    public BankAccount(string accountNumber, string
accountHolder, decimal initialBalance)
    {
        this.accountNumber = accountNumber;
        this.accountHolder = accountHolder;
        this.balance = initialBalance;
    }

    public void Deposit(decimal amount)
    {
        balance += amount;
        Console.WriteLine($"Deposited {amount:C} into account
{accountNumber}");
    }

    public void Withdraw(decimal amount)
    {
        if (balance >= amount)
        {
            balance -= amount;
            Console.WriteLine($"Withdrawn {amount:C} from
account {accountNumber}");
        }
        else
        {
            Console.WriteLine($"Insufficient funds in account
{accountNumber}");
        }
    }
}
```



```
public void DisplayBalance()
{
    Console.WriteLine($"Account {accountNumber} balance:
{balance:C}");
}
}
```

3. În fișierul "Program.cs", adăugați următorul cod în metoda **Main**:

```
BankAccount account = new BankAccount("1234567890", "John Doe",
1000);
account.DisplayBalance();
account.Deposit(500);
account.DisplayBalance();
account.Withdraw(200);
account.DisplayBalance();
account.Withdraw(1500);
account.DisplayBalance();
```

4. Rulați programul și observați outputul generat.
5. Modificați clasa **BankAccount** pentru a adăuga o proprietate read-only **AccountNumber** și o metodă **TransferFunds** care permite transferul de fonduri între două conturi bancare.

Teme pentru acasă

1. Creați o clasă **Student** cu proprietăți precum **Name**, **Grade** și **StudentId**. Adăugați metode pentru a calcula media notelor și pentru a afișa detaliile studentului.
2. Implementați o clasă **Book** cu proprietăți precum **Title**, **Author** și **ISBN**. Adăugați o metodă pentru a afișa informațiile despre carte într-un format specific.
3. Creați o clasă **Employee** cu proprietăți precum **Name**, **Position** și **Salary**. Adăugați metode pentru a calcula salariul anual și pentru a mări salariul cu un anumit procent.
4. Implementați o clasă **Point** care reprezintă un punct în sistemul de coordonate 2D. Adăugați metode pentru a calcula distanța dintre două puncte și pentru a determina poziția unui punct față de un alt punct (stânga, dreapta, sus, jos).

Laboratorul 5: Colecții și LINQ în C#

Obiective

În cadrul acestui laborator, studenții vor:

- Înțelege și utiliza diferite tipuri de colecții în C#, cum ar fi tablouri, liste și dicționare
- Efectua operații comune pe colecții, cum ar fi adăugarea, ștergerea și căutarea elementelor
- Utiliza expresii LINQ pentru a interoga și manipula datele din colecții
- Rezolva probleme practice folosind colecții și LINQ

Activități

Tablouri și liste

În această activitate veți explora utilizarea tablourilor și listelor în C#.

Pași de urmat:

1. Creați un fișier nou numit "TablouriSiListe.cs" în folderul laboratorului.
2. Adăugați următorul cod în fișier:

```
using System;
using System.Collections.Generic;

class Program
{
    static void Main(string[] args)
    {
        // Array-uri
        int[] numbers = { 1, 2, 3, 4, 5 };
        Console.WriteLine("Array-ul numbers:");
        foreach (int number in numbers)
        {
            Console.WriteLine(number);
        }

        string[] fruits = new string[3];
        fruits[0] = "apple";
        fruits[1] = "banana";
        fruits[2] = "orange";
        Console.WriteLine("\nArray-ul fruits:");
        foreach (string fruit in fruits)
        {
            Console.WriteLine(fruit);
        }
    }
}
```

```

    }

    // Liste
    List<int> numbersList = new List<int>();
    numbersList.Add(1);
    numbersList.Add(2);
    numbersList.Add(3);
    Console.WriteLine("\nLista numbersList:");
    foreach (int number in numbersList)
    {
        Console.WriteLine(number);
    }

    List<string> fruitsList = new List<string> { "apple",
"banana", "orange" };
    Console.WriteLine("\nLista fruitsList:");
    foreach (string fruit in fruitsList)
    {
        Console.WriteLine(fruit);
    }
}
}

```

3. Rulați programul și observați outputul generat.
4. Modificați codul pentru a adăuga elemente noi în tablouri și liste, pentru a le accesa și pentru a le elimina.

Dicționare

În această activitate veți explora utilizarea dicționarelor în C#.

Pașii de urmat:

1. Creați un fișier nou numit "Dicționare.cs" în folderul laboratorului.
2. Adăugați următorul cod în fișier:

```

using System;
using System.Collections.Generic;

class Program
{
    static void Main(string[] args)
    {
        Dictionary<string, int> ages = new Dictionary<string,
int>();
        ages["Alice"] = 25;
        ages["Bob"] = 30;
        ages["Charlie"] = 35;
    }
}

```

```
        Console.WriteLine("Dicționar ages:");
        foreach (KeyValuePair<string, int> entry in ages)
        {
            Console.WriteLine($"{entry.Key}: {entry.Value}");
        }

        Dictionary<string, string> capitals = new
Dictionary<string, string>
        {
            ["România"] = "București",
            ["Franța"] = "Paris",
            ["Germania"] = "Berlin"
        };

        Console.WriteLine("\nDicționar capitals:");
        foreach (KeyValuePair<string, string> entry in capitals)
        {
            Console.WriteLine($"{entry.Key}: {entry.Value}");
        }
    }
}
```

3. Rulați programul și observați outputul generat.
4. Modificați codul pentru a adăuga și a elimina elemente din dicționare, pentru a verifica existența unei chei și pentru a accesa valorile corespunzătoare cheilor.

Interogări LINQ

În această activitate veți utiliza expresii LINQ pentru a interoga colecții.

Pașii de urmat:

1. Creați un fișier nou numit "InterogariLINQ.cs" în folderul laboratorului.
2. Adăugați următorul cod în fișier:

```
using System;
using System.Collections.Generic;
using System.Linq;

class Program
{
    static void Main(string[] args)
    {
        List<int> numbers = new List<int> { 1, 2, 3, 4, 5, 6, 7,
8, 9, 10 };

        // Filtrare
        var evenNumbers = from number in numbers
                           where number % 2 == 0
```

```

        select number;
        Console.WriteLine("Numere pare:");
        foreach (int number in evenNumbers)
        {
            Console.WriteLine(number);
        }

        // Sortare
        var sortedNumbers = from number in numbers
                            orderby number descending
                            select number;
        Console.WriteLine("\nNumere sortate descrescător:");
        foreach (int number in sortedNumbers)
        {
            Console.WriteLine(number);
        }

        // Grupare
        var numberGroups = from number in numbers
                            group number by number % 5 into g
                            select new { Remainder = g.Key,
Numbers = g };
        Console.WriteLine("\nGrupare după restul împărțirii la
5:");
        foreach (var group in numberGroups)
        {
            Console.WriteLine($"Restul: {group.Remainder}");
            foreach (int number in group.Numbers)
            {
                Console.WriteLine(number);
            }
        }
    }
}

```

3. Rulați programul și observați outputul generat.
4. Modificați codul pentru a efectua alte interogări LINQ, cum ar fi proiecție, concatenare și agregare.

Manipularea datelor cu LINQ

În această activitate veți utiliza LINQ pentru a manipula date dintr-o colecție.

Pașii de urmat:

1. Creați un fișier nou numit "ManipulareDate.cs" în folderul laboratorului.
2. Adăugați următorul cod în fișier:

```
using System;
```

```
using System.Collections.Generic;
using System.Linq;

class Student
{
    public string Name { get; set; }
    public int Grade { get; set; }
}

class Program
{
    static void Main(string[] args)
    {
        List<Student> students = new List<Student>
        {
            new Student { Name = "Alice", Grade = 85 },
            new Student { Name = "Bob", Grade = 70 },
            new Student { Name = "Charlie", Grade = 90 },
            new Student { Name = "David", Grade = 75 },
            new Student { Name = "Eve", Grade = 95 }
        };

        // Filtrare studenți cu nota peste 80
        var goodStudents = from student in students
                           where student.Grade > 80
                           select student;
        Console.WriteLine("Studenți cu nota peste 80:");
        foreach (Student student in goodStudents)
        {
            Console.WriteLine($"{student.Name}:
{student.Grade}");
        }

        // Calcularea mediei notelor
        var averageGrade = students.Average(student =>
student.Grade);
        Console.WriteLine($"{\nMedia notelor: {averageGrade}");

        // Găsirea studentului cu nota maximă
        var topStudent = (from student in students
                           orderby student.Grade descending
                           select student).First();
        Console.WriteLine($"{\nStudentul cu nota maximă:
{topStudent.Name}");

        // Gruparea studenților după notă
        var studentGroups = from student in students
```

```

                                group student by student.Grade into
g
                                select new { Grade = g.Key, Students
= g };
    Console.WriteLine("\nGruparea studenților după notă:");
    foreach (var group in studentGroups)
    {
        Console.WriteLine($"Nota: {group.Grade}");
        foreach (Student student in group.Students)
        {
            Console.WriteLine(student.Name);
        }
    }
}

```

3. Rulați programul și observați outputul generat.
4. Modificați codul pentru a efectua alte operații de manipulare a datelor, cum ar fi sortarea studenților după nume, calcularea numărului de studenți cu note într-un anumit interval și adăugarea de noi studenți în colecție.

Teme pentru acasă

1. Creați o aplicație care gestionează o listă de angajați. Fiecare angajat are un nume, o funcție și un salariu. Utilizați LINQ pentru a efectua operații precum filtrarea angajaților după funcție, calcularea salariului mediu și găsirea angajatului cu cel mai mare salariu.
2. Implementați o aplicație care gestionează un inventar de produse. Fiecare produs are un nume, o categorie și un preț. Utilizați LINQ pentru a efectua operații precum gruparea produselor după categorie, calcularea prețului mediu pentru fiecare categorie și găsirea celui mai scump și celui mai ieftin produs din fiecare categorie.
3. Creați o aplicație care gestionează o bibliotecă de cărți. Fiecare carte are un titlu, un autor și un gen literar. Utilizați LINQ pentru a efectua operații precum filtrarea cărților după gen, sortarea cărților după autor și calcularea numărului de cărți scrise de fiecare autor.
4. Implementați o aplicație care gestionează un registru de cursuri și studenți. Fiecare curs are un nume și un profesor, iar fiecare student are un nume și o listă de cursuri la care participă. Utilizați LINQ pentru a efectua operații precum găsirea studenților înscriși la un anumit curs, calcularea numărului de cursuri pentru fiecare student și identificarea profesorilor cu cei mai mulți studenți.

Laboratorul 6: Gestionarea erorilor în C#

Obiective

În cadrul acestui laborator, studenții vor:

- Înțelege conceptele de bază ale gestionării erorilor în C#
- Utiliza blocuri try-catch pentru a prinde și a trata excepțiile
- Familiarizarea cu diferite tipuri de excepții și clase de excepții
- Arunca și propaga excepții folosind instrucțiunea throw
- Crea și utiliza excepții personalizate
- Aplicarea unor bune practici în gestionarea erorilor

Activități

Prinderea și tratarea excepțiilor cu try-catch

În această activitate veți utiliza blocuri try-catch pentru a prinde și a trata excepțiile.

Pași de urmat:

1. Creați un fișier nou numit "TryCatch.cs" în folderul laboratorului.
2. Adăugați următorul cod în fișier:

```
using System;

class Program
{
    static void Main(string[] args)
    {
        try
        {
            Console.Write("Introduceți un număr: ");
            int number = int.Parse(Console.ReadLine());
            Console.WriteLine($"Numărul introdus este:
{number}");
        }
        catch (FormatException)
        {
            Console.WriteLine("Eroare: Formatul introdus nu este
valid.");
        }
        catch (OverflowException)
        {
            Console.WriteLine("Eroare: Numărul introdus este
prea mare sau prea mic.");
        }
    }
}
```



```

        catch (Exception ex)
        {
            Console.WriteLine($"Eroare: {ex.Message}");
        }
        finally
        {
            Console.WriteLine("Bloc finally executat.");
        }
    }
}

```

3. Rulați programul și introduceți diferite valori pentru a observa comportamentul blocurilor try-catch.
4. Modificați codul pentru a adăuga alte blocuri catch pentru a trata alte tipuri de excepții, cum ar fi `DivideByZeroException` și `ArgumentException`.

Aruncarea și propagarea excepțiilor

În această activitate veți arunca și propaga excepții folosind instrucțiunea `throw`.

Pași de urmat:

1. Creați un fișier nou numit "ThrowException.cs" în folderul laboratorului.
2. Adăugați următorul cod în fișier:

```

using System;

class Program
{
    static void Main(string[] args)
    {
        try
        {
            Console.Write("Introduceți un număr pozitiv: ");
            int number = int.Parse(Console.ReadLine());
            ValidateNumber(number);
            Console.WriteLine($"Numărul introdus este:
{number}");
        }
        catch (ArgumentException ex)
        {
            Console.WriteLine($"Eroare: {ex.Message}");
        }
    }

    static void ValidateNumber(int number)
    {
        if (number < 0)
        {

```

```
        throw new ArgumentException("Numărul trebuie să fie  
pozitiv.");  
    }  
}  
}
```

3. Rulați programul și introduceți diferite valori pentru a observa comportamentul instrucțiunii throw.
4. Modificați codul pentru a adăuga o nouă metodă care aruncă o excepție personalizată în anumite condiții.

Crearea și utilizarea excepțiilor personalizate

În această activitate veți crea și utiliza excepții personalizate.

Pași de urmat:

1. Creați un fișier nou numit "CustomException.cs" în folderul laboratorului.
2. Adăugați următorul cod în fișier:

```
using System;  
  
class InvalidAgeException : Exception  
{  
    public InvalidAgeException(string message) : base(message)  
    {  
    }  
}  
  
class Person  
{  
    private string name;  
    private int age;  
  
    public Person(string name, int age)  
    {  
        this.name = name;  
        ValidateAge(age);  
        this.age = age;  
    }  
  
    private void ValidateAge(int age)  
    {  
        if (age < 0 || age > 120)  
        {  
            throw new InvalidAgeException("Vârsta trebuie să fie  
între 0 și 120.");  
        }  
    }  
}
```

```

        public void DisplayInfo()
        {
            Console.WriteLine($"Nume: {name}, Vârstă: {age}");
        }
    }

    class Program
    {
        static void Main(string[] args)
        {
            try
            {
                Person person1 = new Person("John", 30);
                person1.DisplayInfo();

                Person person2 = new Person("Alice", -5);
                person2.DisplayInfo();
            }
            catch (InvalidAgeException ex)
            {
                Console.WriteLine($"Eroare: {ex.Message}");
            }
        }
    }
}

```

3. Rulați programul și observați comportamentul excepției personalizate `InvalidAgeException`.
4. Modificați codul pentru a adăuga o nouă clasă de excepție personalizată și utilizați-o într-o altă parte a programului.

Aplicarea bunelor practici în gestionarea erorilor

În această activitate veți aplica bune practici în gestionarea erorilor.

Pașii de urmat:

1. Creați un fișier nou numit "ErrorHandlingBestPractices.cs" în folderul laboratorului.
2. Adăugați următorul cod în fișier:

```

using System;
using System.IO;

class Program
{
    static void Main(string[] args)
    {
        try

```

```
        {
            string filePath = "data.txt";
            ProcessFile(filePath);
        }
        catch (FileNotFoundException ex)
        {
            Console.WriteLine($"Eroare: Fișierul nu a fost
găsit. {ex.Message}");
            // Înregistrarea erorii în log sau raportarea către
utilizator
        }
        catch (Exception ex)
        {
            Console.WriteLine($"Eroare: {ex.Message}");
            // Înregistrarea erorii în log sau raportarea către
echipa de dezvoltare
        }
    }

    static void ProcessFile(string filePath)
    {
        try
        {
            string content = File.ReadAllText(filePath);
            Console.WriteLine($"Conținutul fișierului:
{content}");
        }
        catch (IOException ex)
        {
            Console.WriteLine($"Eroare: Nu s-a putut citi
fișierul. {ex.Message}");
            throw; // Propagarea excepției către apelant
        }
    }
}
```

3. Rulați programul și observați cum sunt gestionate diferite tipuri de excepții și cum sunt aplicate bunele practici, cum ar fi înregistrarea erorilor și propagarea excepțiilor.
4. Modificați codul pentru a adăuga o nouă metodă care citește date dintr-un fișier și aplicați bunele practici de gestionare a erorilor.

Teme pentru acasă

1. Creați o aplicație care efectuează operații aritmetice pe baza intrărilor de la utilizator. Gestionați excepțiile pentru intrări nevalide, cum ar fi împărțirea la zero și depășirea limitelor de valori întregi.

2. Implementați o aplicație care citește informații despre un angajat (nume, vârstă, salariu) din fișiere separate. Gestionați excepțiile pentru fișiere lipsă, formate nevalide și valori incorecte ale datelor.
3. Creați o aplicație care procesează un fișier CSV și extrage informații specifice. Gestionați excepțiile pentru fișiere corupte, formate incorecte și coloane lipsă.
4. Implementați o aplicație care se conectează la o bază de date și efectuează operații CRUD (*Create, Read, Update, Delete*). Gestionați excepțiile pentru conexiuni eșuate, interogări invalide și erori de restricții de cheie străină.

Laboratorul 7: Prelucrarea fișierelor CSV și XML cu LINQ în C#

Obiective

În cadrul acestui laborator, studenții vor:

- Înțelege structura și utilizarea fișierelor CSV și XML
- Utiliza LINQ pentru a interoga și procesa date din fișiere CSV și XML
- Aplica operații de filtrare, proiecție și agregare pe date folosind LINQ
- Rezolva probleme practice ce implică prelucrarea fișierelor CSV și XML

Activități

Prelucrarea fișierelor CSV cu LINQ

În această activitate veți utiliza LINQ pentru a citi și procesa date dintr-un fișier CSV.

Pașii de urmat:

1. Creați un fișier nou numit "Persons.csv" în folderul laboratorului cu următorul conținut:

```
Id,FirstName,LastName,Email,Age
1,John,Doe,john@example.com,30
2,Jane,Doe,jane@example.com,25
3,Bob,Smith,bob@example.com,40
```

2. Creați un fișier nou numit "CsvProcessing.cs" în folderul laboratorului.
3. Adăugați următorul cod în fișier:

```
using System;
using System.Collections.Generic;
using System.IO;
using System.Linq;

class Person
{
    public int Id { get; set; }
    public string FirstName { get; set; }
    public string LastName { get; set; }
    public string Email { get; set; }
    public int Age { get; set; }
}

class Program
```

```
{
    static void Main(string[] args)
    {
        string csvFilePath = "Persons.csv";
        List<Person> persons = ReadCsvFile(csvFilePath);

        // Query 1: Selectați toate persoanele cu vârsta mai
mare de 30 de ani
        var query1 = from person in persons
                      where person.Age > 30
                      select person;

        Console.WriteLine("Persoane cu vârsta mai mare de 30 de
ani:");
        foreach (var person in query1)
        {
            Console.WriteLine($"{person.FirstName}
{person.LastName}, {person.Age}");
        }

        // Query 2: Grupați persoanele după vârstă și numărați-
le
        var query2 = from person in persons
                      group person by person.Age into ageGroup
                      select new { Age = ageGroup.Key, Count =
ageGroup.Count() };

        Console.WriteLine("\nNumărul de persoane pentru fiecare
vârstă:");
        foreach (var group in query2)
        {
            Console.WriteLine($"Vârsta {group.Age}:
{group.Count} persoane");
        }
    }

    static List<Person> ReadCsvFile(string csvFilePath)
    {
        List<Person> persons = new List<Person>();

        using (StreamReader reader = new
StreamReader(csvFilePath))
        {
            // Ignorăm prima linie (header-ul)
            reader.ReadLine();

            string line;
```

```
        while ((line = reader.ReadLine()) != null)
        {
            string[] fields = line.Split(',');
            Person person = new Person
            {
                Id = int.Parse(fields[0]),
                FirstName = fields[1],
                LastName = fields[2],
                Email = fields[3],
                Age = int.Parse(fields[4])
            };
            persons.Add(person);
        }
    }

    return persons;
}
```

4. Rulați programul și observați rezultatele interogărilor LINQ pe datele din fișierul CSV.
5. Adăugați o nouă interogare LINQ care selectează numele complet (FirstName și LastName) și e-mailul persoanelor cu vârsta cuprinsă între 20 și 40 de ani.

Prelucrarea fișierelor XML cu LINQ

În această activitate veți utiliza LINQ to XML pentru a citi și procesa date dintr-un fișier XML.

Pașii de urmat:

1. Creați un fișier nou numit "Books.xml" în folderul laboratorului cu următorul conținut:

```
<?xml version="1.0" encoding="utf-8"?>
<books>
  <book>
    <title>Book 1</title>
    <author>Author 1</author>
    <year>2020</year>
  </book>
  <book>
    <title>Book 2</title>
    <author>Author 2</author>
    <year>2019</year>
  </book>
  <book>
    <title>Book 3</title>
```



```
<author>Author 1</author>
<year>2021</year>
</book>
<book>
  <title>Book 4</title>
  <author>Author 3</author>
  <year>2018</year>
</book>
</books>
```

2. Creați un fișier nou numit "XmlProcessing.cs" în folderul laboratorului.
3. Adăugați următorul cod în fișier:

```
using System;
using System.Linq;
using System.Xml.Linq;

class Program
{
    static void Main(string[] args)
    {
        string xmlFilePath = "Books.xml";
        XDocument xmlDoc = XDocument.Load(xmlFilePath);

        // Query 1: Selectați toate titlurile cărților
        var query1 = from book in xmlDoc.Descendants("book")
                     select book.Element("title").Value;

        Console.WriteLine("Titlurile cărților:");
        foreach (var title in query1)
        {
            Console.WriteLine(title);
        }

        // Query 2: Numărul de cărți pentru fiecare autor
        var query2 = from book in xmlDoc.Descendants("book")
                     group book by book.Element("author").Value
                     into authorGroup
                     select new { Author = authorGroup.Key,
                                   Count = authorGroup.Count() };

        Console.WriteLine("\nNumărul de cărți pentru fiecare autor:");
        foreach (var group in query2)
        {
            Console.WriteLine($"{group.Author}: {group.Count} cărți");
        }
    }
}
```

```
}  
}  
}
```

4. Rulați programul și observați rezultatele interogărilor LINQ pe datele din fișierul XML.
5. Adăugați o nouă interogare LINQ care selectează titlurile și anii cărților publicate după anul 2019.

Teme pentru acasă

1. Creați un fișier CSV care conține informații despre produse (nume, categorie, preț) și utilizați LINQ pentru a calcula prețul mediu al produselor pentru fiecare categorie.
2. Având un fișier XML care conține date despre angajați (nume, departament, salariu), utilizați LINQ to XML pentru a afișa numele și salariul angajaților din departamentul "IT".
3. Folosind un fișier CSV care conține date despre studenți (nume, notă1, notă2, notă3), utilizați LINQ pentru a calcula media notelor pentru fiecare student și afișați numele și media obținută.
4. Creați un fișier XML care descrie o bibliotecă de filme (titlu, gen, an) și utilizați LINQ to XML pentru a afișa titlurile filmelor din genul "Comedy" lansate după anul 2010.

Laboratorul 8: Introducere în F#

Obiective

În cadrul acestui laborator, studenții vor:

- Înțelege sintaxa de bază a limbajului F#
- Explora principalele tipuri de date în F#
- Defini și utiliza funcții simple
- Efectua exerciții practice pentru a se familiariza cu F#

Activități

Sintaxa de bază F#

În această activitate veți explora elementele fundamentale ale sintaxei F# și veți crea primul dumneavoastră program F#.

Pașii de urmat:

1. Deschideți Visual Studio Code și creați un folder nou pentru laboratorul 8.
2. Creați un fișier nou numit "SintaxaBaza.fs" în folderul laboratorului.
3. Adăugați următorul cod în fișier:

```
// Definirea unei funcții
let salut nume =
    printfn "Salut, %s!" nume

// Apelarea funcției
salut "Ana"
salut "Mihai"

// Definirea unei liste
let numere = [1; 2; 3; 4; 5]

// Iterarea printr-o listă folosind o expresie de tip "list
comprehension"
let patrate = [for n in numere do yield n * n]

// Afișarea rezultatului
printfn "Numerele inițiale: %A" numere
printfn "Pătratele numerelor: %A" patrate
```

4. Salvați fișierul și rulați programul folosind comanda `dotnet fsi SintaxaBaza.fs` în terminal. Observați rezultatele.
5. Analizați codul și identificați următoarele elemente de sintaxă:
 - a. Definirea unei funcții folosind cuvântul cheie `let`

- b. Apelarea unei funcții
- c. Definirea unei liste folosind paranteze pătrate []
- d. Utilizarea unei expresii de tip "list comprehension" pentru a itera printr-o listă
- e. Afișarea rezultatelor folosind funcția `printfn`

Tipuri de date în F#

În această activitate veți explora principalele tipuri de date în F# și veți efectua operații pe acestea.

Pași de urmat:

1. Creați un fișier nou numit "TipuriDeDate.fs" în folderul laboratorului.
2. Adăugați următorul cod în fișier:

```
// Tipuri de date simple
let intreg = 42
let flotent = 3.14
let sir = "Hello, F#!"
let boolean = true

// Tipul tuple
let persoana = ("Ana", 25)

// Tipul list
let numere = [1; 2; 3; 4; 5]

// Tipul option
let existaValoare = Some 42
let valoareLipsa = None

// Tipul record
type Student = { Nume: string; Varsta: int }
let student = { Nume = "Mihai"; Varsta = 22 }

// Tipul discriminated union
type Forma =
    | Cerc of raza: float
    | Dreptunghi of lungime: float * latime: float
    | Patrat of latura: float

let cerc = Cerc 5.0
let dreptunghi = Dreptunghi (4.0, 3.0)
let patrat = Patrat 2.5

// Afișarea valorilor
printfn "Număr întreg: %i" intreg
```

```
printfn "Număr flotant: %f" flotant
printfn "Șir de caractere: %s" sir
printfn "Valoare booleană: %b" boolean
printfn "Persoana: %A" persoana
printfn "Lista de numere: %A" numere
printfn "Valoare existentă: %A" existaValoare
printfn "Valoare lipsă: %A" valoareLipsa
printfn "Student: %s, %d ani" student.Nume student.Varsta
printfn "Cerc cu raza %f" (match cerc with Cerc r -> r | _ ->
0.0)
printfn "Dreptunghi cu lungimea %f și lățimea %f"
    (match dreptunghi with Dreptunghi (l, w) -> l, w | _ -> 0.0,
0.0)
```

3. Rulați programul folosind comanda `dotnet fsi TipuriDeDate.fs` și observați rezultatele.
4. Analizați codul și identificați următoarele tipuri de date:
 - a. Tipuri simple: `int`, `float`, `string`, `bool`
 - b. Tipul `tuple` pentru a reprezenta perechi de valori
 - c. Tipul `list` pentru a reprezenta colecții ordonate de valori
 - d. Tipul `option` pentru a reprezenta valori care pot exista sau nu
 - e. Tipul `record` pentru a defini structuri de date cu câmpuri numite
 - f. Tipul `discriminated union` pentru a reprezenta valori care pot avea una dintre mai multe forme

Funcții în F#

În această activitate veți defini și utiliza funcții în F#, explorând concepte precum aplicarea funcțiilor, funcții anonime și funcții recursive.

Pașii de urmat:

1. Creați un fișier nou numit "Funcții.fs" în folderul laboratorului.
2. Adăugați următorul cod în fișier:

```
// Definirea unei funcții cu doi parametri
let aduna x y = x + y

// Aplicarea funcției
let suma = aduna 10 20
printfn "Suma: %i" suma

// Funcție anonimă (lambda)
let numerePare = List.filter (fun x -> x % 2 = 0) [1..10]
printfn "Numere pare: %A" numerePare

// Funcție recursivă pentru calculul factorialului
let rec factorial n =
```

```
if n <= 1 then 1
else n * factorial (n - 1)

let rezultat = factorial 5
printfn "Factorialul lui 5: %i" rezultat
```

3. Rulați programul folosind comanda `dotnet fsi Functii.fs` și observați rezultatele.
4. Analizați codul și identificați următoarele elemente:
 - a. Definirea unei funcții cu doi parametri folosind cuvântul cheie `let`
 - b. Aplicarea unei funcții prin specificarea numelui funcției urmat de valorile parametrilor
 - c. Utilizarea unei funcții anonime (lambda) pentru a filtra o listă
 - d. Definirea unei funcții recursive pentru a calcula factorialul unui număr

Teme pentru acasă

1. Scrieți o funcție F# care primește o listă de numere întregi și returnează suma pătratelor numerelor pare din listă.
2. Definiți un tip record `Dreptunghi` cu câmpurile `Lungime` și `Latime`. Scrieți o funcție care primește o listă de dreptunghiuri și returnează aria totală a acestora.
3. Utilizați o expresie de tip "list comprehension" pentru a genera o listă cu primele 20 de numere din șirul lui Fibonacci.
4. Definiți un tip discriminated union `Optiune` cu două cazuri: `Unele de 'a` și `Nimic`. Scrieți o funcție care primește o listă de valori de tip `Optiune` și returnează o listă cu valorile ne-`Nimic`.
5. Scrieți o funcție recursivă care primește un număr întreg și returnează suma cifrelor sale.

Laboratorul 9: Programare funcțională în F#

Obiective

În cadrul acestui laborator, studenții vor:

- Înțelege și aplica conceptul de pattern matching în F#
- Implementa funcții recursive pentru a rezolva probleme
- Utiliza funcții de ordin superior pentru a manipula colecții de date
- Aplica paradigma de programare funcțională în rezolvarea unor probleme specifice

Activități

Pattern matching

În această activitate veți explora conceptul de pattern matching în F# și veți vedea cum poate fi utilizat pentru a controla fluxul de execuție al programului în funcție de structura datelor.

Pași de urmat:

1. Creați un fișier nou numit "PatternMatching.fs" în folderul laboratorului.
2. Adăugați următorul cod în fișier:

```
// Definirea unui tip de date algebric
type Forme =
    | Cerc of raza: float
    | Dreptunghi of lungime: float * latime: float
    | Patrat of latura: float

// Funcție care calculează aria unei forme geometrice folosind
// pattern matching
let ariaForme forma =
    match forma with
    | Cerc raza -> System.Math.PI * raza * raza
    | Dreptunghi (lungime, latime) -> lungime * latime
    | Patrat latura -> latura * latura

// Crearea unor forme geometrice
let cerc = Cerc 5.0
let dreptunghi = Dreptunghi (4.0, 3.0)
let patrat = Patrat 2.5

// Calcularea ariilor folosind funcția ariaForme
let ariaCerc = ariaForme cerc
let ariaDreptunghi = ariaForme dreptunghi
```

```
let ariaPatrat = ariaForme patrat

// Afișarea rezultatelor
printfn "Aria cercului: %.2f" ariaCerc
printfn "Aria dreptunghiului: %.2f" ariaDreptunghi
printfn "Aria pătratului: %.2f" ariaPatrat
```

3. Rulați programul folosind comanda `dotnet fsi PatternMatching.fs` și observați rezultatele.
4. Analizați codul și identificați următoarele elemente:
 - a. Definirea unui tip de date algebric `Forme` care reprezintă diferite forme geometrice
 - b. Utilizarea expresiei `match` pentru a realiza pattern matching pe tipul `Forme`
 - c. Extragerea datelor din fiecare caz al tipului algebric în funcție de pattern-ul specificat
 - d. Calcularea ariei pentru fiecare formă geometrică în funcție de pattern-ul căruia îi corespunde

Recursivitate

În această activitate veți implementa funcții recursive pentru a rezolva probleme care se pretează la o abordare recursivă.

Pași de urmat:

1. Creați un fișier nou numit "Recursivitate.fs" în folderul laboratorului.
2. Adăugați următorul cod în fișier:

```
// Funcție recursivă pentru calcularea sumei elementelor unei  
liste  
let rec sumaLista lista =  
    match lista with  
    | [] -> 0  
    | cap :: coada -> cap + sumaLista coada  
  
// Funcție recursivă pentru calcularea celui mai mare divizor  
comun (CMMDC) a două numere  
let rec cmmdc a b =  
    if b = 0 then a  
    else cmmdc b (a % b)  
  
// Funcție recursivă pentru generarea șirului lui Fibonacci până  
la un număr dat  
let rec fibonacci n =  
    match n with  
    | 0 -> 0  
    | 1 -> 1
```



```
| _ -> fibonacci (n - 1) + fibonacci (n - 2)

// Apelarea funcțiilor recursive
let numere = [1; 2; 3; 4; 5]
let suma = sumaLista numere
printfn "Suma elementelor listei %A este %d" numere suma

let a = 24
let b = 18
let rezultatCMMDC = cmmdc a b
printfn "CMMDC al numerelor %d și %d este %d" a b rezultatCMMDC

let n = 10
let sirFibonacci = List.init n (fun i -> fibonacci i)
printfn "Șirul lui Fibonacci până la %d este %A" n sirFibonacci
```

3. Rulați programul folosind comanda `dotnet fsi Recursivitate.fs` și observați rezultatele.
4. Analizați codul și identificați următoarele elemente:
 - a. Definirea de funcții recursive pentru calcularea sumei elementelor unei liste, determinarea CMMDC a două numere și generarea șirului lui Fibonacci
 - b. Utilizarea expresiei `match` pentru a identifica cazurile de bază și cazurile recursive
 - c. Apelul recursiv al funcțiilor pentru a rezolva subproblemele mai mici
 - d. Utilizarea funcției `List.init` pentru a genera o listă cu primele `n` numere din șirul lui Fibonacci

Funcții de ordin superior

În această activitate veți utiliza funcții de ordin superior pentru a manipula colecții de date într-un stil funcțional.

Pași de urmat:

1. Creați un fișier nou numit "FuncțiiOrdinSuperior.fs" în folderul laboratorului.
2. Adăugați următorul cod în fișier:

```
// Funcție de ordin superior pentru a aplica o transformare
fiecărui element al unei liste
let aplicaTransformare transformare lista =
    lista |> List.map transformare

// Funcție de ordin superior pentru a filtra elementele unei
liste care satisfac un predicat
let filtreazaElemente predicat lista =
    lista |> List.filter predicat
```

```
// Funcție de ordin superior pentru a calcula suma pătratelor  
numerelor impare dintr-o listă  
let sumaPairPatrate lista =  
    lista  
    |> List.filter (fun x -> x % 2 = 0)  
    |> List.map (fun x -> x * x)  
    |> List.sum  
  
// Apelarea funcțiilor de ordin superior  
let numere = [1; 2; 3; 4; 5]  
  
let dublare x = x * 2  
let numereDublate = aplicaTransformare dublare numere  
printfn "Numerele dublate sunt %A" numereDublate  
  
let numerePare = filtreazaElemente (fun x -> x % 2 = 0) numere  
printfn "Numerele pare sunt %A" numerePare  
  
let suma = sumaPairPatrate numere  
printfn "Suma pătratelor numerelor impare este %d" suma
```

3. Rulați programul folosind comanda `dotnet fsi` `FuncțiiOrdinSuperior.fs` și observați rezultatele.
4. Analizați codul și identificați următoarele elemente:
 - a. Definirea de funcții de ordin superior care primesc alte funcții ca parametri
 - b. Utilizarea operatorului pipe (`|>`) pentru a aplica o serie de operații asupra unei liste
 - c. Folosirea funcțiilor `List.map`, `List.filter` și `List.sum` pentru a manipula și procesa elementele unei liste
 - d. Compunerea funcțiilor pentru a realiza operații complexe într-un mod concis și expresiv

Teme pentru acasă

1. Scrieți o funcție recursivă care primește un număr întreg și returnează suma cifrelor sale.
2. Implementați o funcție care primește o listă de numere întregi și returnează o listă cu pătratele numerelor impare folosind funcția `List.map`.
3. Definiți un tip algebric `Expresie` care reprezintă expresii aritmetice simple (de exemplu, `Numar of int`, `Adunare of Expresie * Expresie`, `Inmultire of Expresie * Expresie`). Scrieți o funcție care evaluează o expresie folosind pattern matching.
4. Scrieți o funcție de ordin superior care primește o listă și o funcție de agregare (de exemplu, `+`, `*`) și returnează rezultatul agregării elementelor listei folosind funcția specificată.

5. Implementați algoritmul Quick Sort folosind o abordare funcțională în F#.

Laboratorul 10: Tipuri de date în F#

Obiective

În cadrul acestui laborator, studenții vor:

- Înțelege și utiliza tipuri de date record în F#
- Defini și folosi tipuri de date discriminate (discriminated unions)
- Explora tipul Option pentru reprezentarea valorilor opționale
- Aplica tipurile de date algebrice în modelarea și rezolvarea problemelor

Activități

Tipuri record

În această activitate veți defini și utiliza tipuri de date record pentru a modela entități din lumea reală.

Pași de urmat:

1. Creați un fișier nou numit "RecordTypes.fs" în folderul laboratorului.
2. Adăugați următorul cod în fișier:

```
// Definirea unui tip record pentru a reprezenta o persoană
type Persoana = {
    Nume: string
    Varsta: int
    Adresa: string
}

// Crearea unei instanțe a tipului record
let persoana1 = { Nume = "Ion Popescu"; Varsta = 30; Adresa =
"Str. Mihai Eminescu, Nr. 10" }

// Accesarea câmpurilor tipului record
printfn "Numele persoanei este %s" persoana1.Nume
printfn "Vârsta persoanei este %d" persoana1.Varsta
printfn "Adresa persoanei este %s" persoana1.Adresa

// Actualizarea unei instanțe a tipului record
let persoana2 = { persoana1 with Varsta = 31 }
printfn "Noua vârstă a persoanei este %d" persoana2.Varsta
```

3. Rulați programul folosind comanda `dotnet fsi RecordTypes.fs` și observați rezultatele.
4. Analizați codul și identificați următoarele elemente:

- Definirea unui tip record **Persoana** cu câmpurile **Nume**, **Varsta** și **Adresa**
- Crearea unei instanțe a tipului record folosind sintaxa **{ Nume = ...; Varsta = ...; Adresa = ... }**
- Accesarea câmpurilor unei instanțe a tipului record folosind notația punct (de exemplu, **persoana1.Nume**)
- Actualizarea unei instanțe a tipului record folosind sintaxa **{ persoana1 with Varsta = ... }**

Tipuri discriminate (discriminated unions)

În această activitate veți defini și utiliza tipuri de date discriminate pentru a modela entități cu variante distincte.

Pașii de urmat:

1. Creați un fișier nou numit "DiscriminatedUnions.fs" în folderul laboratorului.
2. Adăugați următorul cod în fișier:

```
// Definirea unui tip discriminat pentru a reprezenta forme geometrice
type Forma =
    | Cerc of raza: float
    | Dreptunghi of lungime: float * latime: float
    | Triunghi of baza: float * inaltime: float

// Funcție pentru a calcula aria unei forme geometrice
let aria forma =
    match forma with
    | Cerc raza -> System.Math.PI * raza * raza
    | Dreptunghi (lungime, latime) -> lungime * latime
    | Triunghi (baza, inaltime) -> 0.5 * baza * inaltime

// Crearea unor instanțe ale tipului discriminat
let cerc = Cerc 5.0
let dreptunghi = Dreptunghi (4.0, 3.0)
let triunghi = Triunghi (6.0, 4.0)

// Calcularea ariei formelor geometrice
printfn "Aria cercului este %.2f" (aria cerc)
printfn "Aria dreptunghiului este %.2f" (aria dreptunghi)
printfn "Aria triunghiului este %.2f" (aria triunghi)
```

3. Rulați programul folosind comanda **dotnet fsi DiscriminatedUnions.fs** și observați rezultatele.
4. Analizați codul și identificați următoarele elemente:
 - a. Definirea unui tip discriminat **Forma** cu variantele **Cerc**, **Dreptunghi** și **Triunghi**

- b. Utilizarea pattern matching-ului pentru a identifica varianta specifică a tipului discriminat și a extrage datele asociate
- c. Calcularea ariei pentru fiecare variantă a tipului discriminat în funcție de datele extrase
- d. Crearea de instanțe ale tipului discriminat folosind constructorii specifici fiecărei variante (de exemplu, **Cerc 5.0**)

Tipul Option

În această activitate veți explora utilizarea tipului Option pentru a reprezenta valori care pot fi prezente sau absente.

Pașii de urmat:

1. Creați un fișier nou numit "OptionType.fs" în folderul laboratorului.
2. Adăugați următorul cod în fișier:

```
// Funcție pentru a căuta un element într-o listă
let cautaElement element lista =
    let rezultat = List.tryFind (fun x -> x = element) lista
    match rezultat with
    | Some valoare -> printfn "Elementul %A a fost găsit în listă" valoare
    | None -> printfn "Elementul %A nu a fost găsit în listă" element

// Exemplu de utilizare
let lista = [1; 2; 3; 4; 5]

cautaElement 3 lista
cautaElement 6 lista

// Funcție pentru a diviza două numere
let divide numarator numitor =
    if numitor = 0 then
        None
    else
        Some (numarator / numitor)

// Exemplu de utilizare
let rezultat1 = divide 10 2
let rezultat2 = divide 10 0

match rezultat1 with
| Some valoare -> printfn "Rezultatul diviziei este %f" valoare
| None -> printfn "Divizia nu poate fi efectuată"

match rezultat2 with
```

```
| Some valoare -> printfn "Rezultatul diviziei este %f" valoare
| None -> printfn "Divizia nu poate fi efectuată"
```

1. Rulați programul folosind comanda `dotnet fsi OptionType.fs` și observați rezultatele.
2. Analizați codul și identificați următoarele elemente:
 - a. Utilizarea funcției `List.tryFind` pentru a căuta un element într-o listă, care returnează un rezultat de tip `Option`
 - b. Pattern matching pe tipul `Option` pentru a trata cazurile `Some` (când valoarea este prezentă) și `None` (când valoarea este absentă)
 - c. Definirea unei funcții `divide` care returnează un rezultat de tip `Option` pentru a reprezenta cazurile în care împărțirea este posibilă sau nu
 - d. Utilizarea tipului `Option` pentru a evita erorile de împărțire la zero și pentru a trata explicit cazurile de valori absente

Teme pentru acasă

1. Definiți un tip record `Angajat` cu câmpurile `Nume`, `Departament` și `Salariu`. Creați o listă de angajați și calculați media salariilor.
2. Definiți un tip discriminat `Vehicul` cu variantele `Masina`, `Bicicleta` și `Camion`. Scrieți o funcție care primește o listă de vehicule și returnează numărul total de roți (considerând că o mașină are 4 roți, o bicicletă are 2 roți și un camion are 6 roți).
3. Scrieți o funcție care primește o listă de numere întregi și returnează un tuple `Option` cu cel mai mare și cel mai mic element din listă (dacă lista este vidă, returnați `None`).
4. Definiți un tip discriminat `Expr` pentru a reprezenta expresii aritmetice simple (de exemplu, `Numar of int`, `Adunare of Expr * Expr`, `Inmultire of Expr * Expr`). Scrieți o funcție care evaluează o expresie și returnează un rezultat de tip `Option` (dacă expresia este invalidă, returnați `None`).
5. Creați o funcție care primește o listă de șiruri de caractere și un șir de caractere `prefix`. Returnați o listă cu toate șirurile care încep cu `prefix`, folosind funcția `List.filter` și tipul `Option`.

Laboratorul 11: Colecții și prelucrarea datelor în F#

Obiective

În cadrul acestui laborator, studenții vor:

- Înțelege și utiliza colecțiile imutabile în F#, cum ar fi listele și secvențele
- Aplica operații de bază pe colecții, cum ar fi maparea, filtrarea și reducerea
- Utiliza expresii de tip "list comprehension" pentru a genera colecții
- Explora tipurile Map și Set pentru a lucra cu dicționare și seturi
- Rezolva probleme de prelucrare a datelor folosind funcționalitățile oferite de colecții

Activități

Liste și secvențe

În această activitate veți explora utilizarea listelor și secvențelor pentru a lucra cu colecții de date în F#.

Pași de urmat:

1. Creați un fișier nou numit "ListeSecvente.fs" în folderul laboratorului.
2. Adăugați următorul cod în fișier:

```
// Crearea unei liste
let lista = [1; 2; 3; 4; 5]

// Accesarea elementelor dintr-o listă
let primulElement = List.head lista
let restulElementelor = List.tail lista

// Concatenarea listelor
let lista1 = [1; 2; 3]
let lista2 = [4; 5; 6]
let listaConcat = List.concat [lista1; lista2]

// Generarea unei secvențe
let secventa = seq { 1..10 }

// Aplicarea operațiilor pe secvențe
let secventaPare = Seq.filter (fun x -> x % 2 = 0) secventa
let secventaPatrate = Seq.map (fun x -> x * x) secventa
let secventaSuma = Seq.sum secventa

// Afișarea rezultatelor
printfn "Primul element din listă: %d" primulElement
```



```
printfn "Restul elementelor din listă: %A" restulElementelor
printfn "Lista concatenată: %A" listaConcat
printfn "Secvența generată: %A" secventa
printfn "Secvența elementelor pare: %A" secventaPare
printfn "Secvența pătratelor: %A" secventaPatrate
printfn "Suma elementelor din secvență: %d" secventaSuma
```

3. Rulați programul folosind comanda `dotnet fsi ListeSecvente.fs` și observați rezultatele.
4. Analizați codul și identificați următoarele elemente:
 - a. Crearea unei liste folosind sintaxa `[element1; element2; ...]`
 - b. Accesarea primului element și a restului elementelor dintr-o listă folosind funcțiile `List.head` și `List.tail`
 - c. Concatenarea listelor folosind funcția `List.concat`
 - d. Generarea unei secvențe folosind sintaxa `seq { expresie }`
 - e. Aplicarea operațiilor de filtrare (`Seq.filter`), mapare (`Seq.map`) și reducere (`Seq.sum`) pe secvențe

Operații pe colecții

În această activitate veți aprofunda utilizarea operațiilor de bază pe colecții, cum ar fi maparea, filtrarea și reducerea.

Pașii de urmat:

1. Creați un fișier nou numit "OperatiiColecții.fs" în folderul laboratorului.
2. Adăugați următorul cod în fișier:

```
// Definirea unei liste de numere întregi
let numere = [1; 2; 3; 4; 5; 6; 7; 8; 9; 10]

// Maparea numerelor la pătratele lor
let patrate = List.map (fun x -> x * x) numere

// Filtrarea numerelor pare
let numerePare = List.filter (fun x -> x % 2 = 0) numere

// Reducerea numerelor prin însumarea lor
let suma = List.fold (fun acc x -> acc + x) 0 numere

// Verificarea dacă toate numerele sunt mai mari decât 5
let toateMaiMariDecat5 = List.forall (fun x -> x > 5) numere

// Verificarea dacă există numere mai mici decât 3
let existaNumereSubTrei = List.exists (fun x -> x < 3) numere

// Calcularea mediei numerelor
```

```
let media = (List.sum numere |> float) / (List.length numere |> float)

// Afișarea rezultatelor
printfn "Numerele inițiale: %A" numere
printfn "Pătratele numerelor: %A" patrate
printfn "Numerele pare: %A" numerePare
printfn "Suma numerelor: %d" suma
printfn "Toate numerele sunt mai mari decât 5? %b" toateMaiMariDecat5
printfn "Există numere mai mici decât 3? %b" existaNumereSubTrei
printfn "Media numerelor: %.2f" media
```

3. Rulați programul folosind comanda `dotnet fsi OperatiiColectii.fs` și observați rezultatele.
4. Analizați codul și identificați următoarele elemente:
 - a. Maparea numerelor la pătratele lor folosind funcția `List.map`
 - b. Filtrarea numerelor pare folosind funcția `List.filter`
 - c. Reducerea numerelor prin însumarea lor folosind funcția `List.fold`
 - d. Verificarea unor condiții pentru toate elementele (`List.forall`) sau pentru cel puțin un element (`List.exists`)
 - e. Calcularea mediei numerelor folosind funcțiile `List.sum` și `List.length`

List comprehensions

În această activitate veți utiliza expresii de tip "list comprehension" pentru a genera colecții în F#.

Pașii de urmat:

1. Creați un fișier nou numit "ListComprehensions.fs" în folderul laboratorului.
2. Adăugați următorul cod în fișier:

```
// Generarea unei liste de pătrate folosind list comprehension
let patrate = [for i in 1..10 do yield i * i]

// Generarea unei liste de numere pare folosind list comprehension cu condiție
let numerePare = [for i in 1..20 do if i % 2 = 0 then yield i]

// Generarea unei liste de tuple folosind list comprehension cu două generatoare
let combinatii = [for i in 1..3 do for j in 4..6 do yield (i, j)]

// Generarea unei liste de șiruri formate folosind list comprehension
```

```
let siruriFibonacci = [for i in 0..10 do yield $"F{i} = {fib i}"]

// Funcția pentru calcularea termenilor din șirul lui Fibonacci
let rec fib n =
    match n with
    | 0 | 1 -> n
    | _ -> fib (n - 1) + fib (n - 2)

// Afișarea rezultatelor
printfn "Pătratele numerelor de la 1 la 10: %A" patrate
printfn "Numerele pare de la 1 la 20: %A" numerePare
printfn "Combinatii de numere: %A" combinatii
printfn "Șirurile Fibonacci: %A" siruriFibonacci
```

3. Rulați programul folosind comanda `dotnet fsi ListComprehensions.fs` și observați rezultatele.
4. Analizați codul și identificați următoarele elemente:
 - a. Generarea unei liste de pătrate folosind expresia `[for i in 1..10 do yield i * i]`
 - b. Generarea unei liste de numere pare folosind list comprehension cu condiție `if`
 - c. Generarea unei liste de tuple folosind list comprehension cu două generatoare
 - d. Generarea unei liste de șiruri formate folosind list comprehension și interpolare de șiruri
 - e. Definirea unei funcții recursive `fib` pentru calcularea termenilor din șirul lui Fibonacci

Map și Set

În această activitate veți explora utilizarea tipurilor `Map` și `Set` pentru a lucra cu dicționare și seturi în F#.

Pașii de urmat:

1. Creați un fișier nou numit "MapSet.fs" în folderul laboratorului.
2. Adăugați următorul cod în fișier:

```
// Crearea unui dicționar folosind tipul Map
let dictionar = Map.ofList [("a", 1); ("b", 2); ("c", 3)]

// Accesarea valorilor dintr-un dicționar
let valoareA = Map.find "a" dictionar
let existaB = Map.containsKey "b" dictionar

// Actualizarea unui dicționar
```

```
let dictionarNou = Map.add "d" 4 dictionar

// Crearea unui set folosind tipul Set
let set = Set.ofList [1; 2; 3; 4; 5]

// Verificarea apartenenței la un set
let contine3 = Set.contains 3 set

// Operații pe seturi
let setNou = Set.add 6 set
let reuniune = Set.union set (Set.ofList [4; 5; 6; 7])
let intersectie = Set.intersect set (Set.ofList [3; 4; 5; 6])

// Afișarea rezultatelor
printfn "Dicționarul inițial: %A" dictionar
printfn "Valoarea pentru cheia 'a': %d" valoareA
printfn "Există cheia 'b' în dicționar? %b" existaB
printfn "Dicționarul actualizat: %A" dictionarNou
printfn "Setul inițial: %A" set
printfn "Setul conține elementul 3? %b" contine3
printfn "Setul actualizat: %A" setNou
printfn "Reuniunea seturilor: %A" reuniune
printfn "Intersecția seturilor: %A" intersectie
```

3. Rulați programul folosind comanda `dotnet fsi MapSet.fs` și observați rezultatele.
4. Analizați codul și identificați următoarele elemente:
 - a. Crearea unui dicționar folosind funcția `Map.ofList` și o listă de perechi cheie-valoare
 - b. Accesarea valorilor dintr-un dicționar folosind funcția `Map.find` și verificarea existenței unei chei folosind `Map.containsKey`
 - c. Actualizarea unui dicționar folosind funcția `Map.add`
 - d. Crearea unui set folosind funcția `Set.ofList`
 - e. Verificarea apartenenței unui element la un set folosind funcția `Set.contains`
 - f. Efectuarea operațiilor pe seturi, cum ar fi adăugarea unui element (`Set.add`), reuniunea (`Set.union`) și intersecția (`Set.intersect`)

Teme pentru acasă

1. Scrieți o funcție care primește o listă de numere întregi și returnează o listă cu pătratele numerelor impare.
2. Implementați o funcție care primește o listă de cuvinte și returnează numărul total de vocale din toate cuvintele.

3. Scrieți o funcție care primește o listă de numere întregi și returnează produsul numerelor pare.
4. Definiți o funcție care primește o listă de șiruri de caractere și returnează șirul format prin concatenarea elementelor în ordine inversă.
5. Creați o funcție care primește un dicționar cu nume de persoane și vârste, și returnează o listă cu numele persoanelor care au vârsta mai mare decât o valoare dată.

Laboratorul 12: Gestionarea erorilor în F#

Obiective

În cadrul acestui laborator, studenții vor:

- Înțelege conceptul de Railway-oriented programming (ROP) și aplicarea sa în gestionarea erorilor
- Utiliza tipul Result pentru a reprezenta rezultate cu posibile erori
- Implementa fluxuri de prelucrare a datelor folosind computation expressions
- Explora tehnici de validare a datelor folosind tipul Option și tipul Result
- Aplica principiile de gestionare a erorilor în dezvoltarea de aplicații F#

Activități

Railway-oriented programming (ROP)

În această activitate, veți explora conceptul de railway-oriented programming (ROP) și veți implementa un flux de prelucrare a datelor folosind acest stil de programare.

Pași de urmat:

1. Creați un fișier nou numit "ROP.fs" în folderul laboratorului.
2. Adăugați următorul cod în fișier:

```
module ROP

type Result<'a, 'b> =
    | Success of 'a
    | Failure of 'b

let bind f result =
    match result with
    | Success x -> f x
    | Failure err -> Failure err

let switch f =
    f >> Result.mapError f

let validate predicate errorMsg x =
    if predicate x then Success x
    else Failure errorMsg

let divideBy bottom top =
    if bottom = 0 then Failure "Division by zero"
    else Success(top / bottom)
```

```
let parseAge input =
    match System.Int32.TryParse input with
    | true, age -> Success age
    | false, _ -> Failure "Invalid age"

let validateAge age =
    validate (fun x -> x >= 18) "Must be an adult" age

let createPerson name age =
    Success(name, age)

// Example usage
let validationWorkflow input =
    input
    |> parseAge
    |> bind validateAge
    |> bind (fun age -> createPerson "John" age)

let adultJohn = validationWorkflow "30"
let invalidJohn = validationWorkflow "abc"
let youngJohn = validationWorkflow "15"

let mathWorkflow x y z =
    divideBy x y
    |> bind (divideBy z)

let result1 = mathWorkflow 2 10 5
let result2 = mathWorkflow 0 10 5
let result3 = mathWorkflow 2 10 0
```

3. Rulați codul folosind comanda `dotnet fsi ROP.fs` și observați rezultatele.
4. Analizați codul și identificați următoarele elemente:
 - a. Definirea tipului `Result<'a, 'b>` pentru a reprezenta rezultate cu posibile erori
 - b. Implementarea funcțiilor `bind` și `switch` pentru a compune și manipula rezultatele
 - c. Utilizarea funcției `validate` pentru a valida condiții și a genera rezultate de succes sau eroare
 - d. Crearea unui flux de validare `validationWorkflow` folosind compunerea funcțiilor
 - e. Exemplu de flux de prelucrare matematică `mathWorkflow` folosind ROP

Computation Expressions

În această activitate, veți implementa o computation expression pentru a simplifica lucrul cu tipul **Result** și pentru a crea fluxuri de prelucrare a datelor mai expresive.

Pașii de urmat:

1. Creați un fișier nou numit "ResultComputation.fs" în folderul laboratorului.
2. Adăugați următorul cod în fișier:

```
module ResultComputation

type Result<'a, 'b> =
    | Success of 'a
    | Failure of 'b

type ResultBuilder() =
    member _.Bind(m, f) =
        match m with
        | Success x -> f x
        | Failure err -> Failure err

    member _.Return(x) =
        Success x

    member _.ReturnFrom(m) =
        m

let result = ResultBuilder()

// Example usage
let validateName name =
    if String.IsNullOrEmpty name then
        Failure "Name cannot be empty"
    else
        Success name

let validateAge age =
    if age >= 18 then
        Success age
    else
        Failure "Must be an adult"

let createPerson name age =
    result {
        let! validName = validateName name
        let! validAge = validateAge age
        return (validName, validAge)
    }
```



```
let john = createPerson "John" 30
let emptyNameJohn = createPerson "" 30
let youngJohn = createPerson "John" 15
```

1. Rulați codul folosind comanda `dotnet fsi ResultComputation.fs` și observați rezultatele.
2. Analizați codul și identificați următoarele elemente:
 - a. Definirea tipului `Result<'a, 'b>` pentru a reprezenta rezultate cu posibile erori
 - b. Implementarea unei computation expression `ResultBuilder` pentru a simplifica lucrul cu tipul `Result`
 - c. Utilizarea computation expression `result` pentru a crea fluxuri de prelucrare a datelor mai expresive
 - d. Exemplu de utilizare a computation expression pentru validarea și crearea unei persoane

Validarea datelor

În această activitate, veți explora tehnici de validare a datelor folosind tipurile `Option` și `Result` în combinație cu funcțiile de validare.

Pașii de urmat:

1. Creați un fișier nou numit "Validation.fs" în folderul laboratorului.
2. Adăugați următorul cod în fișier:

```
module Validation

let optionValidate predicate errorMsg x =
    if predicate x then Some x
    else None

let resultValidate predicate errorMsg x =
    if predicate x then Ok x
    else Error errorMsg

let parseEmail input =
    if System.String.IsNullOrEmpty input then
        None
    else
        let parts = input.Split('@')
        if parts.Length = 2 && not
            (System.String.IsNullOrEmpty parts.[0]) && not
            (System.String.IsNullOrEmpty parts.[1]) then
            Some input
        else
            None
```

```
let validateAge age =  
    optionValidate (fun x -> x >= 18) "Must be an adult" age  
  
let validatePassword password =  
    resultValidate (fun x -> String.length x >= 8) "Password  
must be at least 8 characters long" password  
  
// Example usage  
let validEmail = parseEmail "john@example.com"  
let invalidEmail = parseEmail "john.example.com"  
  
let validAge = validateAge 30  
let invalidAge = validateAge 15  
  
let validPassword = validatePassword "password123"  
let invalidPassword = validatePassword "pass"
```

3. Rulați codul folosind comanda `dotnet fsi Validation.fs` și observați rezultatele.
4. Analizați codul și identificați următoarele elemente:
 - a. Definirea funcțiilor `optionValidate` și `resultValidate` pentru a valida condiții și a genera rezultate `Option` sau `Result`
 - b. Implementarea unei funcții `parseEmail` pentru a valida formatul unei adrese de email
 - c. Utilizarea funcțiilor de validare `validateAge` și `validatePassword` pentru a valida vârsta și parola
 - d. Exemple de utilizare a funcțiilor de validare cu date valide și invalide

Teme pentru acasă

1. Implementați o funcție `divide` care primește doi întregi și returnează un rezultat `Result<int, string>` reprezentând împărțirea celor două numere. În cazul în care al doilea număr este zero, returnați un mesaj de eroare adecvat.
2. Scrieți o funcție `validatePhoneNumber` care primește un șir de caractere reprezentând un număr de telefon și returnează un rezultat `Option<string>`. Validați formatul numărului de telefon (de exemplu, să conțină doar cifre și să aibă o lungime specifică).
3. Creați o computation expression `optionBuilder` pentru a simplifica lucrul cu tipul `Option`. Utilizați această computation expression pentru a implementa o funcție `createUser` care validează numele de utilizator, parola și adresa de email și returnează un rezultat `Option<User>`, unde `User` este un tip înregistrare.
4. Definiți un tip `Person` cu câmpurile `Name`, `Email` și `Age`. Implementați o funcție `validatePerson` care primește un obiect de tip `Person` și returnează un

rezultat `Result<Person, string list>`, unde `string list` reprezintă o listă de mesaje de eroare. Validați fiecare câmp al obiectului `Person` și colectați toate erorile întâlnite.

5. Scrieți o funcție `parseDate` care primește un șir de caractere reprezentând o dată în formatul "YYYY-MM-DD" și returnează un rezultat `Result<DateTime, string>`. Folosiți funcțiile din modulul `System.DateTime` pentru a parsă data și returnați un mesaj de eroare adecvat în cazul în care formatul datei este invalid.

Laboratorul 13: Prelucrarea fișierelor CSV și XML cu F#

Obiective

În cadrul acestui laborator, studenții vor:

- Înțelege structura și utilizarea fișierelor CSV și XML
- Utiliza funcțiile din modulele **Seq**, **List** și **Array** pentru a procesa date din fișiere CSV
- Folosi biblioteca **FSharp.Data** pentru a citi și interoga date din fișiere XML
- Aplica principiile programării funcționale în prelucrarea datelor
- Rezolva probleme practice ce implică prelucrarea fișierelor CSV și XML

Activități

Prelucrarea fișierelor CSV cu funcții din biblioteca standard

În această activitate veți utiliza funcțiile din modulele **Seq**, **List** și **Array** pentru a citi și procesa date dintr-un fișier CSV.

Pași de urmat:

1. Creați un fișier nou numit "Persons.csv" în folderul laboratorului cu următorul conținut:

```
Id,FirstName,LastName,Email,Age
1,John,Doe,john@example.com,30
2,Jane,Doe,jane@example.com,25
3,Bob,Smith,bob@example.com,40
```

2. Creați un fișier nou numit "CsvProcessing.fsx" în folderul laboratorului.
3. Adăugați următorul cod în fișier:

```
open System.IO

type Person = {
    Id: int
    FirstName: string
    LastName: string
    Email: string
    Age: int
}

let readCsvFile filePath =
    filePath
    |> File.ReadAllLines
    |> Array.skip 1
```

```
|> Array.map (fun line -> line.Split(','))
|> Array.map (fun fields ->
    {
        Id = fields.[0] |> int
        FirstName = fields.[1]
        LastName = fields.[2]
        Email = fields.[3]
        Age = fields.[4] |> int
    })
|> Array.toList

let persons = readCsvFile "Persons.csv"

// Query 1: Selectați toate persoanele cu vârsta mai mare de 30
de ani
let query1 =
    persons
    |> List.filter (fun person -> person.Age > 30)

printfn "Persoane cu vârsta mai mare de 30 de ani:"
query1 |> List.iter (fun person ->
    printfn "%s %s, %d" person.FirstName person.LastName
    person.Age)

// Query 2: Grupați persoanele după vârstă și numărați-le
let query2 =
    persons
    |> List.groupBy (fun person -> person.Age)
    |> List.map (fun (age, persons) -> age, persons |>
List.length)

printfn "\nNumărul de persoane pentru fiecare vârstă:"
query2 |> List.iter (fun (age, count) ->
    printfn "Vârsta %d: %d persoane" age count)
```

4. Rulați codul folosind comanda `dotnet fsi CsvProcessing.fsx` în terminal.
5. Adăugați o nouă interogare care selectează numele complet (FirstName și LastName) și e-mailul persoanelor cu vârsta cuprinsă între 20 și 40 de ani.

Prelucrarea fișierelor XML cu biblioteca FSharp.Data

În această activitate veți utiliza biblioteca `FSharp.Data` pentru a citi și interoga date dintr-un fișier XML.

Pași de urmat:

1. Creați un fișier nou numit "Books.xml" în folderul laboratorului cu următorul conținut:

```
<?xml version="1.0" encoding="utf-8"?>
<books>
  <book>
    <title>Book 1</title>
    <author>Author 1</author>
    <year>2020</year>
  </book>
  <book>
    <title>Book 2</title>
    <author>Author 2</author>
    <year>2019</year>
  </book>
  <book>
    <title>Book 3</title>
    <author>Author 1</author>
    <year>2021</year>
  </book>
  <book>
    <title>Book 4</title>
    <author>Author 3</author>
    <year>2018</year>
  </book>
</books>
```

2. Creați un fișier nou numit "XmlProcessing.fsx" în folderul laboratorului.
3. Adăugați următorul cod în fișier:

```
#r "nuget: FSharp.Data"

open FSharp.Data

type Books = XmlProvider<"Books.xml">

let books = Books.Load("Books.xml")

// Query 1: Selectați toate titlurile cărților
let query1 =
    books.Books
    |> Array.map (fun book -> book.Title)

printfn "Titlurile cărților:"
query1 |> Array.iter (printfn "%s")

// Query 2: Numărul de cărți pentru fiecare autor
let query2 =
    books.Books
    |> Array.groupBy (fun book -> book.Author)
```

```
|> Array.map (fun (author, books) -> author, books |>  
Array.length)  
  
printfn "\nNumărul de cărți pentru fiecare autor:"  
query2 |> Array.iter (fun (author, count) ->  
    printfn "%s: %d cărți" author count)
```

4. Rulați codul folosind comanda `dotnet fsi XmlProcessing.fsx` în terminal.
5. Adăugați o nouă interogare care selectează titlurile și anii cărților publicate după anul 2019.

Teme pentru acasă

1. Creați un fișier CSV care conține informații despre produse (nume, categorie, preț) și utilizați funcțiile din modulele `Seq`, `List` sau `Array` pentru a calcula prețul mediu al produselor pentru fiecare categorie.
2. Având un fișier XML care conține date despre angajați (nume, departament, salariu), utilizați biblioteca `FSharp.Data` pentru a afișa numele și salariul angajaților din departamentul IT.
3. Folosind un fișier CSV care conține date despre studenți (nume, notă1, notă2, notă3), utilizați funcțiile din modulele `Seq`, `List` sau `Array` pentru a calcula media notelor pentru fiecare student și afișați numele și media obținută.
4. Creați un fișier XML care descrie o bibliotecă de filme (titlu, gen, an) și utilizați biblioteca `FSharp.Data` pentru a afișa titlurile filmelor din genul "Comedy" lansate după anul 2010.

Laboratorul 14: Interoperabilitate C# și F#

Obiective

În cadrul acestui laborator, studenții vor:

- Înțelege conceptele de bază ale interoperabilității între limbajele C# și F#
- Utiliza tipuri și funcționalități din C# în codul F#
- Expune tipuri și funcționalități F# către C#
- Dezvolta componente și module utilizând ambele limbaje
- Aplica cele mai bune practici de interoperabilitate în dezvoltarea de aplicații mixte C# și F#

Activități

Utilizarea tipurilor C# în F#

În această activitate, veți explora modalități de a utiliza tipuri și funcționalități din C# în codul F#.

Pași de urmat:

1. Creați o bibliotecă de clase C# numită "CSharpLib" cu următorul cod:

```
using System;

namespace CSharpLib
{
    public class Calculator
    {
        public static int Add(int a, int b)
        {
            return a + b;
        }

        public static int Subtract(int a, int b)
        {
            return a - b;
        }
    }

    public class Person
    {
        public string Name { get; set; }
        public int Age { get; set; }

        public override string ToString()
    }
}
```



```

    {
        return $"Name: {Name}, Age: {Age}";
    }
}

```

2. Creați un proiect F# numit "FSharpApp" și adăugați o referință la biblioteca "CSharpLib".
3. În fișierul "Program.fs", adăugați următorul cod:

```

open System
open CSharpLib

[<EntryPoint>]
let main argv =
    let result = Calculator.Add(10, 20)
    printfn "Result: %d" result

    let person = Person(Name = "John", Age = 30)
    printfn "Person: %0" person

    0

```

4. Rulați aplicația F# și observați rezultatele.
5. Analizați codul și identificați următoarele elemente:
 - a. Utilizarea directivei **open** pentru a accesa namespace-ul și tipurile din biblioteca C#
 - b. Apelarea metodelor statice **Calculator.Add** și **Calculator.Subtract** din clasa C#
 - c. Crearea unei instanțe a clasei **Person** din C# și accesarea proprietăților acesteia
 - d. Utilizarea formatului **%0** în **printfn** pentru a afișa obiecte arbitrare, cum ar fi instanța **Person**

Expunerea tipurilor F# către C#

În această activitate, veți învăța cum să expuneți tipuri și funcționalități F# pentru a fi utilizate în codul C#.

Pași de urmat:

1. În proiectul "FSharpApp", adăugați un fișier nou numit "MathUtils.fs" cu următorul cod:

```

namespace FSharpLib

module MathUtils =
    let square x = x * x

```

```
    let cube x = x * x * x

type Shape =
    | Rectangle of width : float * length : float
    | Circle of radius : float
    | Triangle of base' : float * height : float

module ShapeUtils =
    let area shape =
        match shape with
        | Rectangle (w, l) -> w * l
        | Circle r -> System.Math.PI * r * r
        | Triangle (b, h) -> 0.5 * b * h
```

2. Creați un proiect C# numit "CSharpApp" și adăugați o referință la proiectul "FSharpApp".
3. În fișierul "Program.cs", adăugați următorul cod:

```
using System;
using FSharpLib;

class Program
{
    static void Main(string[] args)
    {
        int result1 = MathUtils.square(5);
        int result2 = MathUtils.cube(3);

        Console.WriteLine($"Square of 5: {result1}");
        Console.WriteLine($"Cube of 3: {result2}");

        var rectangle = new Shape.Rectangle(4.0, 5.0);
        var circle = new Shape.Circle(3.0);
        var triangle = new Shape.Triangle(6.0, 4.0);

        double rectangleArea = ShapeUtils.area(rectangle);
        double circleArea = ShapeUtils.area(circle);
        double triangleArea = ShapeUtils.area(triangle);

        Console.WriteLine($"Rectangle area: {rectangleArea}");
        Console.WriteLine($"Circle area: {circleArea}");
        Console.WriteLine($"Triangle area: {triangleArea}");
    }
}
```

4. Rulați aplicația C# și observați rezultatele.

5. Analizați codul și identificați următoarele elemente:

- Utilizarea funcțiilor **square** și **cube** din modulul **MathUtils** în codul C#
- Crearea instanțelor tipului discriminated union **Shape** din F# în codul C#
- Apelarea funcției **area** din modulul **ShapeUtils** pentru a calcula aria diferitelor forme geometrice

Dezvoltarea de componente mixte C# și F#

În această activitate, veți dezvolta o aplicație care utilizează atât componente C#, cât și F#, demonstrând interoperabilitatea între cele două limbaje.

Pașii de urmat:

- În proiectul "CSharpLib", adăugați o nouă clasă numită "DataProcessor" cu următorul cod:

```
using System;
using System.Collections.Generic;

namespace CSharpLib
{
    public class DataProcessor
    {
        public static List<int> FilterEvenNumbers(List<int>
numbers)
        {
            return numbers.FindAll(x => x % 2 == 0);
        }

        public static List<int> MultiplyNumbers(List<int>
numbers, int multiplier)
        {
            return numbers.ConvertAll(x => x * multiplier);
        }
    }
}
```

- În proiectul "FSharpApp", adăugați un fișier nou numit "DataAnalysis.fs" cu următorul cod:

```
namespace FSharpLib

open CSharpLib

module DataAnalysis =
    let sumNumbers numbers =
        numbers |> List.sum
```

```
let averageNumber numbers =  
    let sum = sumNumbers numbers  
    let count = numbers.Length  
    (sum, count) ||> (/)  
  
let processData numbers =  
    numbers  
    |> DataProcessor.FilterEvenNumbers  
    |> DataProcessor.MultiplyNumbers 2  
    |> sumNumbers
```

3. În fișierul "Program.fs" din proiectul "FSharpApp", actualizați funcția **main** cu următorul cod:

```
[<EntryPoint>]  
let main argv =  
    let numbers = [1; 2; 3; 4; 5; 6; 7; 8; 9; 10]  
  
    let evenNumbers = DataProcessor.FilterEvenNumbers numbers  
    printfn "Even numbers: %A" evenNumbers  
  
    let multipliedNumbers = DataProcessor.MultiplyNumbers  
numbers 3  
    printfn "Multiplied numbers: %A" multipliedNumbers  
  
    let sum = DataAnalysis.sumNumbers numbers  
    printfn "Sum of numbers: %d" sum  
  
    let average = DataAnalysis.averageNumber numbers  
    printfn "Average of numbers: %f" average  
  
    let processedData = DataAnalysis.processData numbers  
    printfn "Processed data: %d" processedData  
  
0
```

4. Rulați aplicația F# și observați rezultatele.
5. Analizați codul și identificați următoarele elemente:
- Utilizarea metodelor **FilterEvenNumbers** și **MultiplyNumbers** din clasa C# **DataProcessor** în codul F#
 - Implementarea funcțiilor **sumNumbers**, **averageNumber** și **processData** în modulul F# **DataAnalysis**
 - Utilizarea pipe-urilor (**|>**) în F# pentru a compune funcții și a crea un flux de prelucrare a datelor
 - Apelarea funcțiilor F# din **DataAnalysis** și a metodelor C# din **DataProcessor** în funcția **main** din F#

Teme pentru acasă

1. Creați o clasă C# numită **StringUtils** cu o metodă statică **ReverseString** care primește un șir de caractere și returnează șirul inversat. Utilizați această metodă într-o aplicație F#.
2. Definiți un tip înregistrare F# numit **Student** cu câmpurile **Name** (șir de caractere), **Grade** (întreg) și **Courses** (listă de șiruri de caractere). Creați o funcție F# **GetHonorStudents** care primește o listă de **Student** și returnează lista studenților cu nota peste 80. Utilizați această funcție într-o aplicație C#.
3. Implementați o clasă C# **BankAccount** cu proprietățile **AccountNumber** (șir de caractere), **Balance** (număr real) și o metodă **Deposit** care primește o sumă și actualizează soldul. Creați o funcție F# **WithdrawFunds** care primește un obiect **BankAccount** și o sumă, și returnează un nou obiect **BankAccount** cu soldul actualizat după retragere. Utilizați această funcție într-o aplicație C#.
4. Definiți un tip discriminated union F# numit **Result<'T, 'E>** cu două cazuri: **Success** și **Failure**. Implementați o funcție F# **TryDivide** care primește doi întregi și returnează un **Result** cu câtul împărțirii (**Success**) sau un mesaj de eroare (**Failure**) în cazul împărțirii la zero. Utilizați această funcție într-o aplicație C#.
5. Creați o clasă C# **Person** cu proprietățile **Name** (șir de caractere) și **Age** (întreg). Definiți un tip modul F# **PersonUtils** cu o funcție **ValidateAge** care primește un obiect **Person** și returnează un **Result<Person, string>**, unde **Success** conține obiectul **Person** dacă vârsta este validă (între 0 și 150), iar **Failure** conține un mesaj de eroare în caz contrar. Utilizați această funcție într-o aplicație C#.